

Rule-based spreadsheet data transformation from arbitrary to relational tables



Alexey O. Shigarov*, Andrey A. Mikhailov

Matrosov Institute for System Dynamics and Control Theory of SB RAS, 134 Lermontov st., Irkutsk 664033, Russia

ARTICLE INFO

Article history:

Received 28 June 2017

Revised 11 August 2017

Accepted 12 August 2017

Available online 14 August 2017

Keywords:

Spreadsheet data transformation

Table understanding

Table model

Table analysis

Table interpretation

Rule-based programming

ABSTRACT

The paper discusses issues of rule-based data transformation from arbitrary spreadsheet tables to a canonical (relational) form. We present a novel table object model and rule-based language for table analysis and interpretation. The model is intended to represent a physical (cellular) and logical (semantic) structure of an arbitrary table in the transformation process. The language allows drawing up this process as consecutive steps of table understanding, i. e. recovering implicit semantics. Both are implemented in our tool for spreadsheet data canonicalization. The presented case study demonstrates the use of the tool for developing a task-specific rule-set to convert data from arbitrary tables of the same genre (government statistical websites) to flat file databases. The performance evaluation confirms the applicability of the implemented rule-set in accomplishing the stated objectives of the application.

© 2017 Elsevier Ltd. All rights reserved.

1. Introduction

Spreadsheets provide a popular way for creating and circulating arbitrary tables (e. g. cross-tabulations, invoices, roadmaps, and data collection forms). They can be considered as a general form for representing tabular data with an explicitly presented layout (cellular structure) and style (graphical formatting). For example, HTML tables presented in web pages can be easily converted to spreadsheet formats. The arbitrary tables can be a valuable data source in business intelligence and data-driven research. However, difficulties that inevitably arise with extraction and integration of the tabular data often hinder the intensive use of them in the mentioned areas.

The number of genuine tables in the Web reaches hundreds of millions [1–3]. Many of them are relational tables that can be considered as flat databases. Nevertheless, there are other popular types of tables [4–7] having layout features designed for human understanding (e. g. merged cells, footnotes, and indentations). These include about 50% of tables presented in 0.4M spreadsheets of ClueWeb09 Crawl¹ [5] and 147M (61%) of 233M web tables extracted from Common Crawl² [3]. They lack explicit seman-

tics required for computer programs to interpret their layout and content.

Table understanding is to recover the missing semantics. The papers [8,9] defines the five consecutive stages of the table understanding: *detection* of a table in a document, *recognition* (segmentation) of its cellular structure, *functional* and *structural analysis* for recovering its logical structure, and *interpretation* that aspire to recover its semantics through linking its content with target schema or domain concepts.

We regard the transformation of spreadsheet tabular data (Fig. 1a) into the relational form (Fig. 1b) as a process of table understanding. In the general case, this transformation includes all the enumerated stages:

1. *Detection*. A spreadsheet document can contain several arbitrary tables surrounded by text and graphics.
2. *Recognition*. A human-readable structure of an arbitrary table can differ from its machine-readable structure presented in a spreadsheet, e. g. one logical cell can be visually composed of several physical cells through drawing their borders.
3. *Role (functional) analysis*. A spreadsheet cell stores a text, where a human can distinguish one or more data items that play some functional roles in a table (e. g. values or attributes). However, there are no spreadsheet metadata that separate data items from a cell value and determine their functional roles.
4. *Structural analysis*. A spreadsheet also contains no metadata for representing relationships between data items of a table.

* Corresponding author.

E-mail addresses: shigarov@icc.ru, shigarov@gmail.com (A.O. Shigarov), mikhailov@icc.ru (A.A. Mikhailov).

¹ <http://lemurproject.org/clueweb09>.

² <http://commoncrawl.org>.

a

CURRENCY	FISCAL YEAR	SALES CHANNEL	Category	Label	Rows
c1			-Retail Sales	c2	1
				Catalog Sales	2
			-FY2016	FY2017	3
			(thousands of dollars)	(thousands of dollars)	4
					5
					6
					7
					8

b

DATA	SALES CHANNEL	FISCAL YEAR	CURRENCY	PRODUCT
11200	retail	2016	u.s. dollars	electronics/phones
23700	retail	2017	u.s. dollars	electronics/phones
12600	catalog	2016	u.s. dollars	electronics/phones
32200	catalog	2017	u.s. dollars	electronics/phones

Fig. 1. A fragment of a source arbitrary table (a); a fragment of a target table in the canonical form generated from the source table (b).

5. *Interpretation.* A data item can be an instance of a concept (category), but its spreadsheet does not explicitly associate it with a domain ontology or a global taxonomy.

The paper covers the rule-based analysis and interpretation of arbitrary tables presented in spreadsheets. Our contribution consists of the following results:

1. We present a novel table object model designed for representing a physical (cellular) and logical (semantic) structure of an arbitrary table in the transformation process (Section 2). Our model associates roles with data items instead of cells or cell regions (e.g. head, stub, or body). Moreover, it provides data provenance for recovered semantics.
2. We propose CRL (Cells Rule Language), a domain-specific language for expressing table analysis and interpretation rules (Section 3). A set of the rules can be implemented for a specific task characterized by requirements for source and target data.
3. We develop TABBYXL, a tool for rule-based transformation of arbitrary tables presented in spreadsheets into the canonical (relational) form (Section 4). The tool implements our table object model and rule language.
4. We evaluate an experimental application that is intended to convert data from tables of the same genre (government statistical websites) to flat file databases (Section 5). It exemplifies the use of our language for developing a task-specific rule-set. The performance evaluation confirms the applicability of the implemented rule-set in accomplishing the stated objectives of this application.

2. Table object model

The table object model is designed for representing both a physical structure and logical data items of an arbitrary table in the process of its analysis and interpretation (Fig. 2). Our model adopts the terminology of Wang's table model [10]. It includes two inter-related layers: *physical* (Section 2.1) represented by the collection of cells and *logical* (Section 2.2) that consists of three collections of entries (values), labels (keys), and categories (concepts). We deliberately resort to the two-way references between the layers to

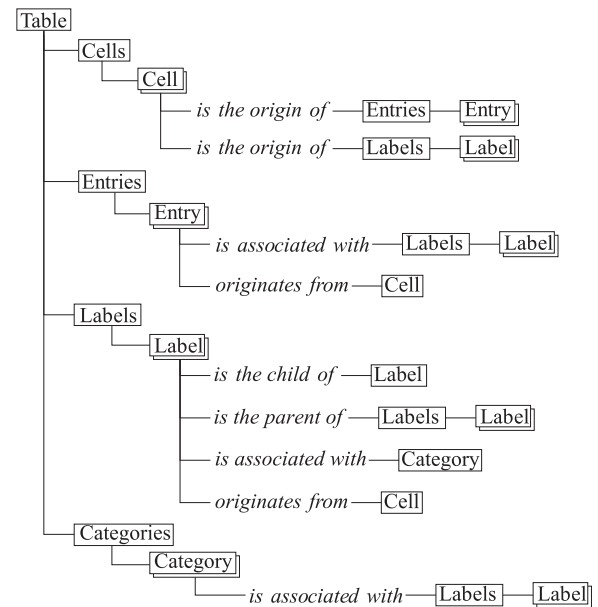


Fig. 2. Two-layered table object model.

provide convenient access to their objects in table analysis and interpretation rules.

2.1. Physical layer

Cell object models common features of a cell that can be presented in tagged documents of well-known formats, such as Excel, Word, or HTML. We define Cell object as a set of the following attributes:

- *Location:* c1 – left column, rt – top row, cr – right column, and rb – bottom row. A cell located on several consecutive rows and columns covers a few grid tiles, which always compose a rectangle. Moreover, two cells cannot overlap each other.
- *Style:* font – font features (name, color, height, etc.), bgColor and fgColor – background and foreground colors, rotation – text rotation, horzAlignment

and `vertAlignment` – horizontal and vertical alignment, `leftBorder`, `topBorder`, `rightBorder`, and `bottomBorder` – border features (types, colors), and `style` encapsulates the listed attributes.

- *Content*: `text` – textual content, `indent` – indent, and `type` – its literal data type (NUMERIC, DATE, STRING, etc.).
- *Annotation*: `mark` – a user-defined word or phrase to annotate the cell.
- *Logical layer references*: `entries` (a set of entries) and `labels` (a set of labels) originated from this cell. Thus, a cell can contain several entries and labels.

For example, an initial state of some cells (`c1,...,c6`) shown in Fig. 1a can be represented as follows:

```
c1=(c1=1,rt=1,cr=1,rb=2,text=null)
c2=(c1=2,rt=1,cr=3,rb=1,text='Retail
Sales')
c3=(c1=2,rt=2,cr=2,rb=2,text='FY2016
(thousands of dollars)')
c4=(c1=1,rt=3,cr=1,rb=3,style.font.bold
=true,text='Electronics')
c5=(c1=1,rt=4,cr=1,rb=4,text='-Phones')
c6=(c1=2,rt=4,cr=2,rb=4,text='11.2',
type=NUMERIC)
```

2.2. Logical layer

Entry object serves to represent a data value of a table. It consists of the following attributes: `value` – a value (text), `labels` – a set of labels associated with this entry, and `cell` – the physical layer reference to a cell as its origin that serves as data provenance. An entry can be associated with only one label in each category.

Label represents a label (key) that addresses one or more entries (data values). It is defined as follows: `value` – a value (text), `children` – a set of labels which are children of this label, `parent` – its parent label, `category` – an associated category, `cell` – the physical layer reference to a cell as its origin (data provenance).

Category models a category of labels as follows: `name` – an internal name, `uri` – a uniform resource identifier representing this category (concept) in an external vocabulary, `labels` – a set of its labels. Each label is associated with only one category. Labels combined into a category can be organized as one or more trees.

This layer allows representing items differently depending on target requirements of the table transformation. For example, one of the possible target can generate the entry (`e1`), labels (`l1,...,l5`), and categories (`d1,...,d4`) for the table shown in Fig. 1a as follows:

```
e1=(value='11200', labels={l1,l2,l3,l5},
cell=c6)
l1=(value='retail', category=d1, cell=c2)
l2=(value='2016', category=d2, cell=c3)
l3=(value='u.s. dollars', category=d3,
cell=c3)
l4=(value='electronics', children={l5,...},
category=d4, cell=c4)
l5=(value='phones', parent=l4, category=d4,
cell=c5)
d1=(name='SALE CHANNEL', labels={l1,...})
d2=(name='FISCAL YEAR', labels={l2,...})
d3=(name='CURRENCY', labels={l3,...})
d4=(name='PRODUCT', labels={l4,l5,...})
```

They can be presented as a tuple of a target relational table (Fig. 1b).

3. Table analysis and interpretation rules

The rules expressed in our language are intended to map explicit features (layout, style, and text of cells) of an arbitrary table into its implicit semantics (entries, labels, and categories) Fig. 3 demonstrates its grammar in Extended Backus–Naur form. A rule begins with the keyword `rule` and ends with `end`. A number that follows the keyword `rule` determines the order of executing this rule. The left hand side (`when`) of a rule consists of one or more *conditions* that enable to query available facts which are cells, entries, labels, and categories of a table. Each of the conditions listed in the left hand side of a rule has to be true to execute its right hand side (`then`) that contains *actions* to modify the existed or to generate new facts about the table.

3.1. Conditions

We use two kinds of conditions. The first requires that there exists at least one fact of a specified data type, which satisfies a set of constraints:

```
cell variable: constraints, assignments
entry variable: constraints, assignments
label variable: constraints, assignments
category variable: constraints, assignments
```

The condition consists of three parts. In its order of occurrence, the first is a keyword which denotes one of the following fact types: `cell`, `entry`, `label`, or `category`. The second is `variable`, a variable of the specified fact type. The third optional part begins with the colon character. It defines constraints for restricting the requested facts and assignments for binding additional variables with values. A *constraint* is a boolean expression in Java. The comma character separating the constraints is the logical conjunction of them. An *assignment* (`variable: value`) sets a value (Java expression) to a variable. A condition without constraints allows querying all facts of specified type.

The second kind of conditions determines that there exist no facts of a specified type, which satisfy a set of constraints:

```
no cells : constraints
no entries : constraints
no labels : constraints
no categories : constraints
```

The first part of these conditions is a keyword for satisfying a type of facts. The second part contains constraints on the facts.

3.2. Cell cleansing

In practice, hand-coded tables often have messy layout (e.g. improperly splitted or merged cells) and content (e.g. typos, homographs, or errors in indents). We address several actions to the issues of cell cleansing, that can be used as the preprocessing stage.

3.2.1. Cell merging

Two cells can be merged when they share one border. The action combines two adjacent cells `$cell1` and `$cell2` into the one merged cell:

```
merge $cell1 with $cell2
```

As a result, the addressee `$cell2` becomes a merged cell with new coordinates that span both cells.

3.2.2. Cell splitting

This action allows dividing a merged cell `$cell` that spans n -tiles into n -cells.

```
split $cell
```

Each of the n -cells completely copies content and style from the merged cell and coordinates from the corresponding tile.

```

rule      = 'rule' <a Java integer literal>
           'when' condition 'then' action 'end' <EOL> {rule} <EOF>
condition = query identifier [':' constraint {',' constraint}
           [',' assignment {',' assignment}]] <EOL> {condition}
constraint = <a Java boolean expression>
assignment = identifier ':' <a valid Java expression>
query      = 'cell' | 'entry' | 'label' | 'category' |
           'no cells' | 'no entries' | 'no labels' | 'no categories'
action     = merge | split | set text | set indent | set mark |
           new entry | new label | add label | set parent |
           set category <EOL> {action}
merge      = 'merge' identifier 'with' identifier
split      = 'split' identifier
set text   = 'set text' <a Java string expression> 'to' identifier
set indent = 'set indent' <a Java integer expression> 'to' identifier
set mark   = 'set mark' <a Java string expression> 'to' identifier
new entry  = 'new entry' identifier ['as' <a Java string expression>]
new label  = 'new label' identifier ['as' <a Java string expression>]
add label  = 'add label' identifier | (<a Java string expression> 'of'
           identifier | <a Java string expression>) 'to' identifier
set parent = 'set parent' identifier 'to' identifier
set category = 'set category' identifier | <a Java string expression>
           'to' identifier
identifier = <a Java identifier>

```

Fig. 3. Grammar of CRL in Extended Backus–Naur form.

			a		b	
			c	d	c	d
e	g		1		2	a
	h	3		4	5	
f	g	6				

			a		b	
			c	d	c	d
e	g	1	1	1	2	b
	h	3	4	4	5	
f	g	6	4	4	5	

Fig. 4. Pivot tables with empty stub heads. We depict here entries as numbers and labels as Latin characters.

Example 1. The table shown in Fig. 4a contains the merged cells ('1', '4', and '5'). We can split them, using the following rule:

```

when
  cell $cc: cl == 1, rt == 1, blank
  cell $c: cl > $cc.cr, rt > $cc.rb
then
  split $c

```

As a result, the table (Fig. 4a) is transformed into the table (Fig. 4b).

3.2.3. Cell content modification

There are two actions modifying cell content. The first sets a new value `string_value` to a cell `$cell`:

```
set text string_value to $cell
```

Some string processing (e.g. regular expressions and string matching algorithms) implemented as Java-methods can be involved in the action.

The second one modifies the indent value `integer_value` of a cell `$cell`:

```
set indent integer_value to $cell
```

3.3. Role analysis

This stage aims to recover entries and labels as functional data items presented in tables. We also enable associating cells with user-defined marks (tags) that can assist in both role and structural analysis.

3.3.1. Cell marking

The action provides marking a cell `$cell` with a word or phrase `string_value`:

```
set mark string_value to $cell
```

The assigned mark can substitute the corresponding constraints in subsequent rules. The typical practice is to set a mark to all cells, which play the same role or are located in the same table functional region. Thereafter, we can use these marks in subsequent rules instead of repeating constraints on cell location in the regions.

Example 2. Looking at the pivot tables shown in Fig. 4, we can assume that each of them has an empty cell (stub head region) located in the top-left corner. It can be considered as a “critical cell” [11] which determines three functional regions: body, head, and stub.

The rule based on the assumption adds the mark (body) to each cell `$c` located in the body.

```

when
  cell $cc: cl == 1, rt == 1, blank
  cell $c: cl > $cc.cr, rt > $cc.rb
then
  set mark 'body' to $c

```

Similarly, we can write rules for marking the corresponding cells with the words: head and stub. As a result of matching these rules against the cells of the table (Fig. 4b), we recover the following facts:

```

c1=(cl=3, rt=3, cr=3, rb=3, value='1',
mark='body'), ...,
c12=(cl=6, rt=5, cr=6, rb=5, value='5',
mark='body'),
c13=(cl=3, rt=1, cr=4, rb=1, value='a',
mark='head'), ...,
c18=(cl=6, rt=2, cr=6, rb=2, value='d',
mark='head'),
c19=(cl=1, rt=3, cr=1, rb=4, value='e',
mark='stub'), ...,
c23=(cl=2, rt=5, cr=2, rb=5, value='g',
mark='stub')

```

3.3.2. Entry and label generating

Two actions presented below generate entries and labels in a cell `$cell`, using string expressions `entry_value` and `label_value` usually obtained as a result of string processing its textual content:

	α a	β b	
γ c	1	2	<i>a</i>
δ d	3	4	

C1	C2	C3	
a = 1	b = 2	c = 3	<i>b</i>
d = 4	e = 5	f = 6	
g = 7	h = 8	i = 9	

Fig. 5. Tables with cells containing more than one entry or label: bilingual table, where each non-empty cell has either two labels or two entries (a); text like as “key=value” in a cell can be interpreted as a label-entry pair, where key-part is a label and value-part is an entry (b).

```
newentry $cell as entry_value
newlabel $cell as label_value
```

The following short form creates an entry and a label from the cell text:

```
newentry $cell
newlabel $cell
```

Example 3. The bilingual table (Fig. 5a) duplicates labels in two languages (Greek and Latin symbols). Assuming that the first label (word) in a cell is written in one language and the second in other, we can use the rule below to generate two labels from each cell located in the leftmost column or the topmost row:

```
when
  cell $c: c1==1 || rt==1, !blank
then
  newlabel $c as token($c, 0)
  newlabel $c as token($c, 1)
```

In this example, we expect that the function `token` is implemented as a Java-method and imported into the rules. It returns a token (word) specified by an index from text of a cell.

For the table (Fig. 5a) this rule generates 8 labels:

```
l1=(value='α'), l2=(value='a'), ...,
l7=(value='δ'), l8=(value='d')
```

For tables similar to the one shown in Fig. 5b, where any cell under the topmost row contains a text as “key=value”, the following rule creates a label from the key-part and an entry from the value-part of the text:

```
when
  cell $c: rt > 1
then
  newlabel $c as left($c, '=')
  newentry $c as right($c, '=')
```

The functions `left` and `right` are implemented as Java-methods. In the presented case, they extract substrings before and after the character (“=”) respectively.

For the table (Fig. 5b) this rule generates 9 entries and 9 labels:

```
e1=(value='1'), ..., e9=(value='9'),
l1=(value='a'), ..., l9=(value='i')
```

3.4. Structural analysis

The next stage recovers pairs of two kinds: entry-label and label-label.

3.4.1. Entry-label associating

The action below binds an entry \$entry with a label \$label:

```
addlabel $label to $entry
```

There are two additional ways to create an entry-label pair. The first associates an entry \$entry with a label specified by its value (label_value) from a category indicated by its name (category_name):

```
addlabel label_value of category_name to $entry
```

The second creates a pair between them similarly but using a defined category \$category:

```
addlabel label_value of $category to $entry
```

In both cases, we try to find or create the label in the specified category.

Example 4. The table in Fig. 6b depicts the use of a cell background color (Color.GRAY³ in the cell B2) as a reference to the footnote (“e”). We can recover this relationship, using the style features as follows:

```
when
  entry $e: cell.style.backgroundColor == Color.GRAY
then
  addlabel 'e' of 'footnotes' to $e
```

We assume that the following facts exist before executing the rule:

```
c1=(style.backgroundColor=Color.GRAY, entries={e1}),
e1=(value='1', cell=c1)
```

As a result, we generate the new facts after its execution:

```
e1=(value='1', cell=c1, labels={l1}),
l1=(value='e', category=d1),
d1=(name='footnotes', labels={l1})
```

3.4.2. Label-label associating

This action connects two labels \$label1 as a parent and \$label2 as its child:

```
setparent $label1 to $label2
```

Example 5. A header located in the stub (leftmost column) often begins with an indent presented as a series of spaces, dots, or other padding characters. Usually, the indents denote hierarchical label-label pairs. For example, when each level in a label hierarchy augments the indents with two additional dots (Fig. 6a), we can recover label-label pairs as follows:

```
when
  cell $c1: c1 == 1
  cell $c2: c1 == 1, rt > $c1.rt, indent == $c1.indent + 2
  no cells : c1 == 1, rt > $c1.rt, rt < $c2.rt, indent == $c1.indent
then
  setparent $c1.label to $c2.label
```

As a result, we recover the following label-label pairs:
(c, c1), (c1, c11), (c1, c12), (c, c2), (c2, c21), (d, d1), (d1, d11)

3.5. Interpretation

The stage includes actions for recovering label-category pairs.

3.5.1. Label categorizing

The action of label categorizing consists in associating a label \$label with a category \$category:

```
setcategory $category to $label
```

Furthermore, a string expression category_name presenting the name of a category can also be used as an argument:

```
setcategory category_name to $label
```

In the latter case, we try to find or create the category with this name.

Example 6. Some tables contain category names among their headings. The stub head of the table shown in Fig. 6c contains two category names: ‘A’ is for the category of the column labels (‘a1’, ‘a2’, and ‘a3’) and ‘B’ is for the category of the row labels (‘b1’ and ‘b2’). The names can be used to create corresponding categories

³ <https://docs.oracle.com/javase/7/docs/api/java/awt/Color.html>.

	a	b
c		
...c1		
...c11	1	2
...c12	3	4
...c2		
...c21	5	6
d		
...d1		
...d11	7	8

a

	a	b
c	1	2
d	3	4

b

— e

	A	a1	a2	a3
B				
b1	1	2	3	
b2	4	5	6	

c

	a	b
c	d	e
f	g	
h	j	2
i	k	1
l		4
		5
		6
		7

d

Fig. 6. The table with indentation in the stub for indicating a row label hierarchy (a); the table where the gray fill color of the cell is used as a reference to the footnote 'e' (b); the table containing two category names: 'A' and 'B' (c); the table with grouped labels (d).

and to categorize labels. In case of tables similar to the one shown in Fig. 6, a, we can assume that a cell in the top-left corner (stub head) contains two category names: the first one describes column labels and the second one addresses row labels.

The rule below creates a category from the first token (word) contained in the top-left corner cell and uses it to categorize column labels:

```
when
  cell $cc: cl == 1, rt == 1
  label $l: cell.rt == 1
then
  set category token($cc, 0) to $l
```

In case of the table (Fig. 6, c), we generate the following facts:

```
l1=(value='a1', category=d1), l2=(value='a2', category=d1),
l3=(value='a3', category=d1), d1=(name='A', labels={l1, l2, l3})
```

3.5.2. Label grouping

Arbitrary tables often place all labels of one category in the same row or column. Consequently, we can suppose that the labels belong to a category without defining its name. In the cases, grouping two or more labels means that they all belong to an undefined category. The action places two labels \$label1 and \$label2 in one group:

```
group $label1 with $label2
```

All labels of a group can be associated with only one category.

Example 7. The stub of the pivot table (Fig. 6d) consists of two columns. We can suppose that all stub labels originated from one column belong to the same undefined category. The rule below arranges its stub labels into two groups ({h, i}) and {j, k, l}):

```
when
  label $l1: cell.mark == 'stub'
  label $l2: cell.mark == 'stub', cell.rt == $l1.cell.rt
then
  group $l1 with $l2
```

4. Implementation

We develop TABBYXL⁴, a command-line tool for spreadsheet data canonicalization that implements our methodology. Its architecture is illustrated in Fig. 7. The data structures for representing table facts (cells, entries, labels, and categories) are Java classes developed in accordance with the naming conventions of JavaBeans⁵ specification. Table facts are instances of these classes. This allows using any rule engine implemented JSR 94: JAVA RULE ENGINE API⁶ specification.

We use DROOLS EXPERT⁷ rule engine. Table analysis and interpretation rules can be expressed in DRL, the native format of DROOLS EXPERT, or CRL, our domain-specific language. CRL rules are preliminarily translated into DRL, using DSL specification with mappings from CRL to DRL constructs. Moreover, CRL rules can be enriched by DRL rule attributes (e.g. for rule activation lock-on-active or no-loop true) and commands for updating the working memory (DRL method update).

The process of table canonicalization begins with loading tabular data from Excel spreadsheets via APACHE POI API⁸. Our tool requires that the pair of tags \$START and \$END point out to location of each input table. It allows avoiding detecting tables in a spreadsheet. Each table forms a set of initial cells. They are asserted as facts in the working memory of the rule engine. The input can also be enriched by categories as facts specified in YAML⁹ format. Each input category consists of its name, a set of labels, and a set of constraints defining ranges of permissible label values [12].

The rule engine matches available facts against loaded rules. As a result, we generate new facts. Recovered semantics (entries, labels, and categories) allows transforming source tabular data into the canonical form. The process ends with that we export each canonical table to spreadsheet format.

The canonical form requires that the topmost row contains field (attribute) names. Each of the remaining rows is a record (tuple). It obligatorily includes the field named DATA that contains entries. Each extracted category constitutes a field that contains its labels. Each record presents recovered relationships between an entry and labels in each category.

4.1. User study

We conducted a user study to evaluate how well potential users can develop CRL rules. We invited 10 practitioners in software engineering, data management, and data analytics to participate in our user study. Each of them had a background of programming in at least one general-purpose language. Only two participants used rule-based languages in their practice.

First, we introduced a tutorial that briefly explained our table object model and demonstrated simple CRL rules for the table understanding stages. Second, we asked each participant to develop a rule-set for transforming source tables of one type (Fig. 8a and c) into the target ones (Fig. 8b and d). The presented source tables were used in a real-world application of collecting information on electrical and technical equipment in a power company.

We required that participants implemented only 7 rules for the following tasks: (1) data cleansing, (2) entry generation, (3) label

⁴ <https://github.com/cellsrg/tabbyxl>.

⁵ <http://www.oracle.com/technetwork/java/javase/tech-spec-136004.html>.

⁶ <http://jcp.org/en/jsr/detail?id=94>.

⁷ <http://drools.jboss.org/drools-expert>.

⁸ <http://poi.apache.org>.

⁹ <http://yaml.org>.

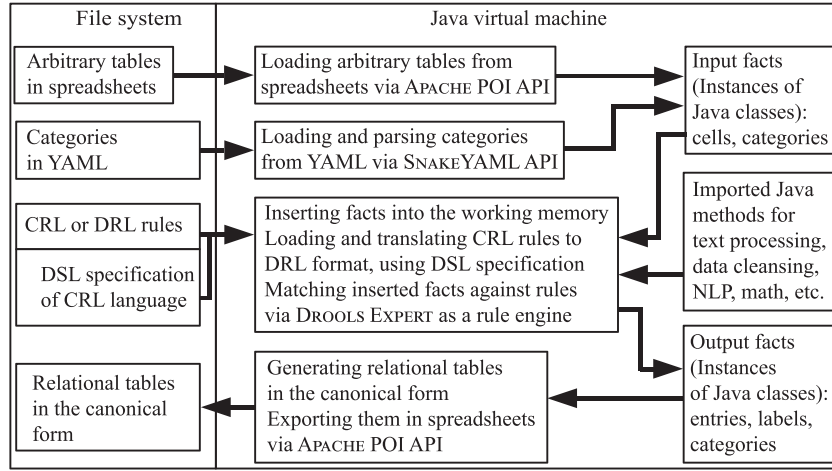


Fig. 7. Architecture of TABBYXL, the tool for spreadsheet data canonicalization.

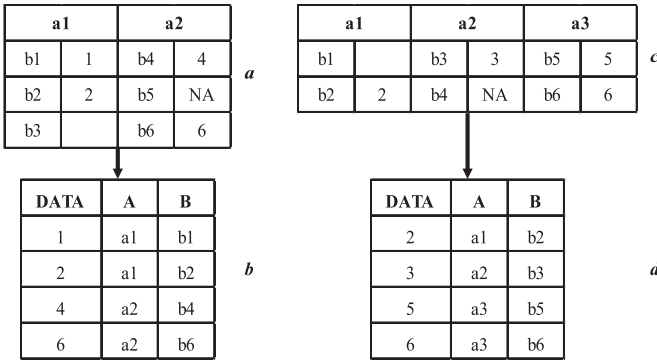


Fig. 8. Source tables (a,c) and their target canonical forms (b,d) used in the user study, that satisfy the following assumptions: $1, \dots, n$ are entries, $a_1, \dots, a_m$ are column labels of the category A , $b_1, \dots, b_k$ are row labels of the category B .

generation, (4) associating entries with column labels, (5) associating entries with row labels, (6) categorizing column labels, and (7) categorizing row labels. The reference rule-set is shown in Fig. 9.

Each participant developed a complete rule-set using a simple text editor. Only 3 participants made 8 syntax errors, and 6 made 14 semantic errors. In several cases, they also used syntactically and semantically correct but redundant constraints.

All errors were made in the part of rule constraints. The syntax errors were the incorrect Java syntax. The participants used the incorrect remainder operator mod instead of % (4 cases), the assignment operator (=) instead of the equality comparison operator (==) (3 cases), and incorrect brackets (1 case). The semantic errors were caused by the following reasons: the missed constraints (`cell.c1==e.cell.c1-1`) required to correctly associate entries with row labels (6 cases); an undefined identifier (4 cases); redundant constraints that prevent to correctly querying data items (2 cases); the use of `c1`-coordinate instead of `cr`-coordinate (2 cases).

In spite of only 3 users developed the rules without errors, the rest participants were able to correct the errors in process of the rule-set compilation and execution. Two participants made about 87% (7/8) of all syntax errors and 64% (9/14) of all semantic errors. The use of a visual editor with syntax highlighting and autocomplete of constraints can render the errors less likely.

We believe that our tool is useful not only for computer scientists in the area of table understanding but also for practitioners in data management and analytics. The user study shows that qualified users such as database administrators or data analysts are able

Table 1

Using cell region layouts in tables of the experiment dataset.

Region	Layout	Fig. 10	Cases
(1) Category name cells	One-column one-row	b	94.5%
	Multi-column one-row	c	5.5%
(2) Column label cells	One-row plain	d	65.5%
	Multi-row plain	e	26%
	Multi-row hierarchical	f	8.5%
(3) Row label cells	One-column hierarchical	g	47.5%
	One-column plain	h	47%
	Multi-column plain	i	5.5%
(4) Entry cells	Multi-column multi-row	j	100%

to design and implement programs (rule-sets) for specific tasks of spreadsheet data transformation.

5. Case study

The purpose of the experiment is to show a possibility of using our tool¹⁰ for tables, which originate from various sources produced by different authors but pertain to the same document genre. The experiment includes two parts: (i) designing and implementing an experimental rule-set for tables of the same genre, and (ii) evaluating the performance of the rule-set on a set of these tables. All data and rules for reproducing the experiment results are available as the published dataset¹¹ [13].

5.1. Dataset

TROY200 [14], the existing dataset of tables, satisfies the purpose of our experiment. It contains 200 arbitrary tables as CSV (Comma-Separated Values) files collected from 10 different sources of the same genre, government statistical websites predominantly presented in English language. We use its earlier version¹² that stores these tables with style features (fonts, alignment, and indentation) as spreadsheets.

Each table of the dataset contains four cell regions, which correspond to three roles: category names, labels, and entries (Fig. 10). A region can have one of the appropriate layouts shown in Fig. 10. Table 1 presents the distribution of these tables by the layouts.

¹⁰ TabbyXL v0.1, <https://github.com/cellsrg/tabbyxl/releases>.

¹¹ <https://data.mendeley.com/datasets/448jdx7gcr/1>.

¹² <http://tango.byu.edu/data>.

- ```

(1) when cell $c: text == "NA"
 then set text "" to $c
(2) when cell $c: (cl % 2) == 0, !blank
 then new entry $c
 when
 entry $e
 label $l: cell.rt == $e.cell.rt, cell.cl == $e.cell.cl - 1
 then add label $l to $e
(3) when cell $c: (cl % 2) == 1
 then new label $c
 when
 entry $e
 label $l: cell.cr == $e.cell.cr
 then add label $l to $e
(4) when label $l: cell.rt == 1
 then set category "A" to $l
(5) when label $l: cell.rt > 1
 then set category "B" to $l

```

Fig. 9. A reference rule-set for transforming the source tables (Fig. 8a and c) to the target canonical forms (Fig. 8b and d).

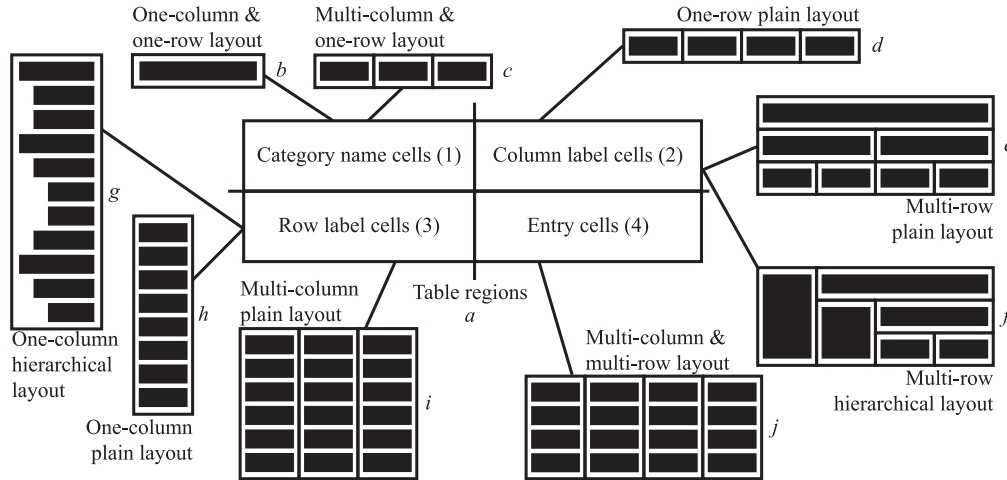


Fig. 10. Cell regions (a) and their layouts (b–j) in tables of the experiment dataset.

## 5.2. Source and target

The role and structural analysis depend on both features of source tables and objectives of table processing. Our examination reveals that most tables presented in the dataset meet the following *source requirements*:

1. A table consists of four cell regions having different functions and separated by two invisible perpendicular lines as shown in Fig. 10a: (i) each non-empty cell of the top-left region contains a category name; (ii) each non-empty cell of the top-right region contains a column label; (iii) each non-empty cell of the bottom-left region contains a row label; (iv) each non-empty cell of the bottom-right region contains an entry.
2. Each cell region has one of the appropriate layouts as shown in Fig. 10 and enumerated in Table 1.
3. Head cells can form a hierarchy of labels when their columns span and nest (Fig. 10e and f).
4. Row labels located in the leftmost column can denote the presence of a label hierarchy: (i) each level of label nesting appends one additional indent equaled two spaces; (ii) hyphen char ('-') at the beginning of a label indicates that the label is nested; (iii) text highlighted by the bold font can signalize spanning label.
5. Data cells contain either numeric values or special words (e.g. '#', 'x').

Our aim consists in transforming the tabular data to a general-purpose target representation that can go through a further interpretation. It provides the following *target requirements*:

1. All labels originated from head cells are placed into one hierarchical category (CalCat) fields in Fig. 11b and d).

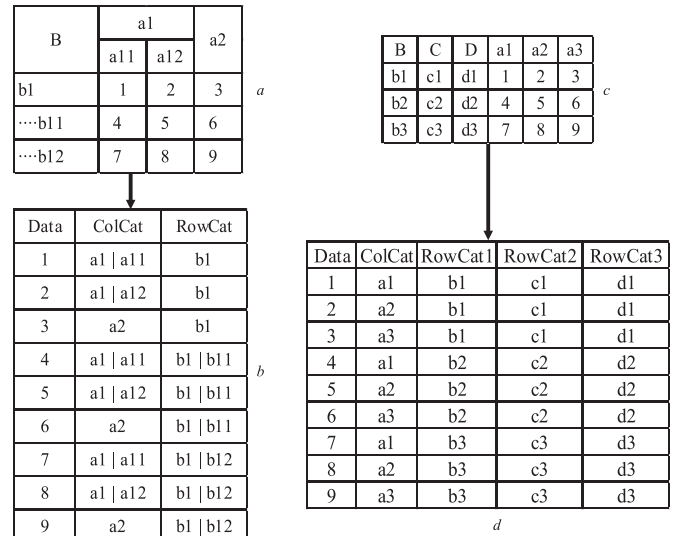


Fig. 11. Two arbitrary tables (a and c) and their canonical forms that satisfy the experimental target requirements (b and d).

2. All labels obtained from the same column of a stub are placed into a separate hierarchical category (RowCat fields in Fig. 11b and d).
3. Footnotes and superscript text are ignored.

## 5.3. Rule-set

We have designed and implemented a rule-set that transforms the tables of the dataset into the canonical form according to the



presented target requirements. It includes 16 rules that can be grouped as follows:

1. *Preprocessing*. Rule-1 replaces each cell value matching “#” or “s” on “0”. Rule-2 replaces each cell value such as “F”, “x”, “NA”, horizontal ellipsis, or dash on null. Rule-3 replaces dashes or hyphens at the beginning of a negative number on the regular minus symbol.
2. *Generating and associating column labels*. We use two rules to generate and associate labels in a head. The first Rule-4 generates labels from non-empty cells of the topmost row. The second Rule-5 searches for head rows top down beginning from the topmost one, examining pairs of neighbor rows. It expects that column labels in the pair form a hierarchy (Fig. 10e and f). If a non-empty cell  $c$  located on  $i$ -row in the top-right region spans several columns and non-empty cells  $c_1, \dots, c_n$  are located in these nested columns on  $i+1$ -row, then the cell  $c$  contains a parent label for labels produced from the cells  $c_1, \dots, c_n$ .
3. *Generating row labels*. Rule-6 creates labels from non-empty cells in the leftmost column. Rule-7 continues to generate labels from cells located below the head in case when their values are not numbers.
4. *Generating entries*. Rule-8 generates entries from numeric values of the remaining cells located below the head.
5. *Associating row labels*. First, we use Rule-9 to fix indentation. It sets up the indent for each cell when its string value begins from a hyphen-minus character. The next two rules recover a hierarchy of row labels located in the leftmost column (Fig. 10g). Rule-10 searches for parent-child pairs among row labels assuming that each new level of a hierarchy appends one additional indent (two spaces) in cells of nested labels. Rule-11 associates a child label with parent one when the parent cell with a boldface text is located above the child cell with a regular text. Exceptions are made for third cases where a parent candidate is “Total”, “All”, or “I alt”. They are ignored.
6. *Categorizing labels*. The three rules (Rule-12, Rule-13 and Rule-14) place labels into synthetic categories: ColCat, RowCat1,..., RowCatN.
7. *Associating entries*. Rule-15 associates an entry with a terminal label when they are originated from cells located in the same column. Rule-16 connects an entry and all labels when their cells are in the same row.

#### 5.4. Performance evaluation

The case study includes the ground-truth data that we have collected from Troy200 tables. The performance evaluation is based on comparing the ground-truth data with the tables generated by executing the presented rule-set for the experiment dataset.

##### 5.4.1. Ground-truth data

The collected ground-truth data covers both role and structural analysis. Their form is designed for human readability. Moreover, they are independent of the presence of critical cells in tables.

Each table from the dataset was accompanied by two recordsets: ENTRIES and LABELS. The first specifies entries. Each record is a triple: an entry, its provenance (cell address in its source table), and a set of associated labels. For example, a fragment of ENTRIES recordset is shown below:

```
ENTRY PROVENANCE LABELS
243 T11 ''2002 [B11]'', ''balance [T4]''
2871 S11 ''2002 [B11]'', ''imports [S4]''
```

Each record in the second recordset presents a label, including a reference to its parent. We demonstrate an example of LABELS recordset below:

```
LABEL PROVENANCE PARENT
```

**Table 2**

Experiment results on the role analysis stage.

|                     | type of instances |        |        |
|---------------------|-------------------|--------|--------|
|                     | entries           | labels | both   |
| Number of           |                   |        |        |
| correct instances   | 16,602            | 4849   | 21,451 |
| existing instances  | 16,918            | 4859   | 21,777 |
| recovered instances | 16,602            | 5178   | 21,780 |
| Recall              | 0.9813            | 0.9979 | 0.9850 |
| Precision           | 0.1000            | 0.9365 | 0.9849 |

**Table 3**

Experiment results on the structural analysis stage.

|                     | type of instances |                   |        |
|---------------------|-------------------|-------------------|--------|
|                     | entry-label pairs | label-label pairs | both   |
| Number of           |                   |                   |        |
| correct instances   | 34,270            | 1951              | 36,221 |
| existing instances  | 35,066            | 2078              | 37,144 |
| recovered instances | 34,386            | 1994              | 36,380 |
| Recall              | 0.9773            | 0.9389            | 0.9751 |
| Precision           | 0.9966            | 0.9784            | 0.9956 |

```
balance T4 other ict goods [R3]
imports S4 other ict goods [R3]
```

#### 5.4.2. Measures

To evaluate our rule-set we adapt the well-known measures: *recall* and *precision*. When  $R$  is a set of instances in the result table and  $S$  is a set of instances in the corresponding source table, then:

$$\text{recall} = \frac{|R \cap S|}{|S|} \quad \text{precision} = \frac{|R \cap S|}{|R|}$$

An instance refers to an entry, label, entry-label pair, or label-label pair. So, these measures can be separately calculated for each type of instances (entries, labels, entry-label pairs, or label-label pairs) as well as for combinations of these types.

#### 5.4.3. Experiment results

To evaluate the role analysis we calculate the recall and precision for entries, labels, and the sum of these both types (Table 2). The performance evaluation at the structural analysis stage is determined as measuring the recall and precision for entry-label pairs, label-label pairs, and pairs of both types (Table 3). The presented results have been obtained automatically through our command-line application included in TABBYXL as `Evaluator.class`.<sup>13</sup>

#### 5.4.4. Errors

Among 200 tables of the dataset, only 25 are processed with errors (1249 false negatives in 25 tables, and 488 false positives in 14 ones). Notice that, one table<sup>14</sup> is not processed. It results in 948 false negative and 316 false positive errors, which amount to about 73% of all errors. Its entries are not recovered because they are not numeric as it is assumed in the used rule-set.

For the rest of cases, the errors came from a stub or head. Their examples are illustrated in Fig. 12. We classify them according to their causes. There are five types of the causes originated from stub (Fig. 12a).

1. *Cut-in*. A header in a row, where all rest cells are empty, can be a root for other headers or just a leaf.

<sup>13</sup> <https://github.com/cellsrg/tabbyxl/blob/master/src/main/java/ru/icc/cells/ssdc/evaluation/Evaluator.java>.

<sup>14</sup> file C10082.csv in Troy200 dataset

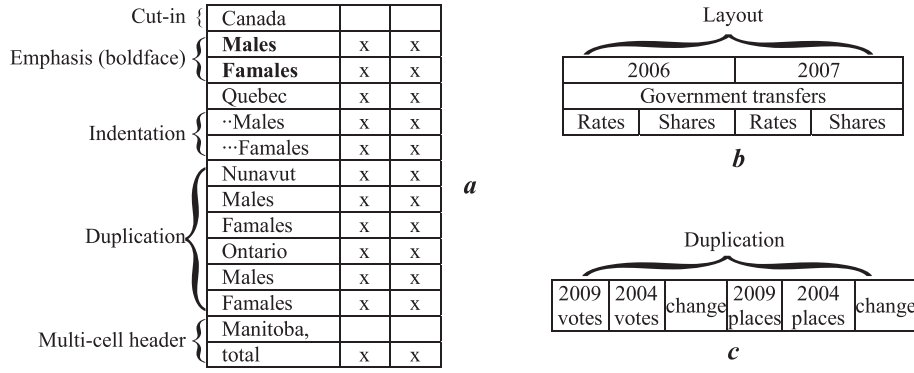


Fig. 12. Causes of errors in stubs(a) and heads(b) and(c).

Table 4

Causes of errors originated from stubs or heads.

| Origin              | Amount of tables | False negatives | False positives |
|---------------------|------------------|-----------------|-----------------|
| Stub                | 21               | 153             | 84              |
| Cut-in              | 10               | 34              | 3               |
| Multi-cell header   | 4                | 48              | 41              |
| Indentation         | 4                | 30              | 14              |
| Emphasis (boldface) | 2                | 30              | 26              |
| Repeating           | 2                | 11              | 0               |
| Head                | 3                | 148             | 88              |
| Layout              | 2                | 130             | 70              |
| Duplication         | 1                | 18              | 18              |
| Total               | 24               | 301             | 172             |

2. *Multi-cell header*. Text parts of a header can be placed into several neighbor cells.
3. *Indentation*. A header hierarchy can be constituted by untidy indents (e.g. padding characters differently aligned).
4. *Emphasis*. The boldface font can be used to emphasize a root in a header hierarchy or data aggregation.
5. *Repeating*. A header hierarchy has one or more sets of repeating headers without visual emphasizing its levels.

The errors from a head are produced for the following causes (Fig. 12b and c).

1. *Layout*. A layout of cells is not expected in our rule-set (e.g. a spanning header located under nested ones).
2. *Duplication*. There are two or more headers with the same text, i.e. additional headers are needed to read them.

Table 4 shows the distribution of the errors that arise in stubs or heads. Among 24 tables processed with errors only one has two types of error causes (“indentation” and “multi-cell header”). Each of the remaining tables is accompanied by only one of the enumerated types.

The presented results show that stubs and heads are complex objects for an automatic analysis and interpretation. On the one hand, the same layout or style features of two stubs or two heads can have different meaning (e.g. the emphasis referencing to a header hierarchy or data aggregation). On the other hand, the absence of the features also leads to errors in the table understanding. Perhaps, these objects require rigorous studying and developing specialized algorithms for their analysis.

## 6. Comparison with others

We experimentally compare our tool with two state of the art solutions for spreadsheet table understanding:

- MIPS (Minimum Indexing Point Search), an algorithm for table segmentation into the functional cell regions, proposed by Em-

bley et al.[15] as a part of their algorithmic solution for end-to-end transforming “header-indexed” tables converted in CSV format to a relational form (“category tables”).

- SENBAZURU, a spreadsheet database management system, proposed by Chen and Cafarella[16,17] for extracting relational data from spreadsheets (“data frames”).

Their goals are similar to ours. Both solutions aim to transform spreadsheet tables with a complex structure into a relational form. They also involve the stages of role and structural analysis. Both rely on similar functional cell regions. Embley et al.[15] define four regions StubHeader (stub head), ColHeader (head), RowHeader (stub), and Data (body). Chen and Cafarella[16,17] focus on data frames, a common data model in spreadsheets. A data frame consists of three regions: *top annotation* (head), *left annotation* (stub) and *data* (body). Both systems also support hierarchical relationships of headings (labels).

The considered systems limit a range of tables, which can be processed successfully, by the assumptions on the functional cell regions. In contrast to these systems, we clearly divide physical and logical table structures on the object level (Section 2). Our model associates roles with data items (entries and labels) instead of cells or regions (e.g. head, stub, or body). Therefore, our tool does not have the limitations of the comparable systems.

We demonstrate the experimental results that are close to ones of the state of the art systems on the existing datasets of real-world tables. Unlike these solutions, our tool allows developing and executing rule-sets to process others types of tables, which do not fit in the presented models (“header-indexed tables” or “data frames”).

### 6.1. Role analysis

We compare our tool with MIPS in role analysis. MIPS exploits the assumption that there exists MIP (Minimum Indexing Point) in a table that determines boundaries of its regions: stub, head, and body. They successfully identify roles of the cell region when MIPS finds MIP correctly.

Embley et al.[15] evaluate only table segmentation into the functional cell regions on Troy200 dataset[14] (briefly described in Section 5.1). They report that one of the 200 Troy200 tables is trivial (it contains only one data column). Among the 199 non-trivial tables, MIP is correctly detected in 197 cases. Therefore, the accuracy of automatic table segmentation (i.e. detection of a stub, head, and body) by MIPS reaches 0.9899 (197/199) on Troy200 dataset.

We evaluate the rule-set presented in Section 5.3. It enables transforming Troy200 tables into the canonical form. We also annotate not blank cells by tags ColumnHeading, RowHeading, and DataCell, which can be considered as tags for cells of a

head, stub, and body respectively. Therefore, we identify which of cells belong to each of these functional regions.

In our experiment, total 7 tables are processed with errors of role analysis. However, only 2 of them are processed with errors, when our rule-set does not detect a head, stub, and body successfully. The *accuracy* of our table segmentation is 0.9950 (198/199) on TROY200 dataset without one trivial table noted by Embley et al. [15]. It amounts 0.9900 (198/200) for all tables.

The presented metrics conditionally show the correctness of role analysis. However, some errors can occur in cases when tables are successfully segmented (Section 5.4.4). We propose to evaluate recall and precision of recovering entries and labels. We demonstrate the following results of recovering entries and labels on TROY200 dataset (Section 5.4.3): *recall* = 0.9850, *precision* = 0.9849, and  $F_1 = 0.9849$ .

## 6.2. Structural analysis

We compare our tool with the automatic extractor of SENBAZURU in structural analysis. SENBAZURU extracts parent-child pairs of labels located in a head (top annotation region) and stub (left annotation region) based on an undirected graphical model. It exploits graphical style features (e.g. font, alignment, and indentation) as well as correlated extraction decisions.

Chen and Cafarella [16,17] evaluated accuracy of the automatic extraction in predicting correct parent-child relationships by using standard metrics of *recall*, *precision*, and  $F_1$ . Their experiment is based on two spreadsheet corpora.<sup>15</sup> One of them is SAUS (The 2010 Statistical Abstract of the United States) that consists of 1369 spreadsheets. They randomly selected 200 tables from SAUS and then split them into two equal parts for training and testing the automatic extractor. They reached the following results: *recall* = 0.8860, *precision* = 0.8860, and  $F_1 = 0.8860$ .

Our experiment exploits a subset of 200 SAUS tables randomly selected and listed by Nagy [14]. We randomly split this subset into two equal parts. The first part was used to develop a rule-set for processing SAUS tables. This rule-set includes 14 rules, 7 of which recover parent-child pairs of labels. We evaluate the rule-set only in recovering parent-child pairs using the second part of the subset.

The process of evaluation is implemented as follows. Two experts independently compare source tables and their automatically generated canonical forms. They examine that each parent-child pair is processed successfully or not. When they make opposite decisions on a pair, then a final decision is a consensus between them. We have obtained the following results: *recall* = 0.7879 (3217/4083), *precision* = 0.9606 (3217/3349), and  $F_1 = 0.8657$ .

Our result ( $F_1$ ) is less only by 0.0233 than one shown by the specialized system SENBAZURU. The many errors caused by inaccurate table markup (physical cell structure) reduce the recall of our tool. There are two main failure reasons in our experiment: (i) in table heads, one human-readable (visual) cell is placed in several physical cells, (ii) in table stubs, the content of one label is distributed in several consecutive cells.

We believe that physical cell structure cleansing (repair) as a preprocessing stage will allow avoiding many of the errors and significantly improve the recall in recovering label-label pairs for tables similar to SAUS ones. We obtain *recall* = 0.9389, *precision* = 0.9784, and  $F_1 = 0.9582$  for recovering parent-child (label-label) pairs on TROY200 dataset (Section 5.4.3), where tables have more accurately markup, i.e. the human-readable (visual) head structure conforms physical one.

## 7. Related work

Many methods and tools, where table analysis and interpretation serve to extract and transform tabular data to a machine-interpretable representation, e.g. relational databases, RDF,<sup>16</sup> or OWL,<sup>17</sup> were suggested in recent years.

### 7.1. Table analysis and interpretation methods

The methods for table analysis are proposed in the papers [5,15,16,18–30]. Each of them is designed for a few widespread types or features of arbitrary tables.

Pivk et al. [18] involve heuristics on structure and textual content of a table, which are designed for three typical table types. Abraham and Erwig [19] combine different heuristic-based algorithms that classify spreadsheet cells into four functional groups (roles). Kim and Lee [20] use the analysis of spatial, style, and textual information from web tables based on embedded rules and regular expressions for five table types. Tao and Embley [21] assume that all content in one cell is either a label or an entry. Several works [15,23,31,32] use the critical cells that allow dividing a table into four functional regions (head, stub, stubhead, and body) [11]. Adelfio and Samet [24] exploit common table patterns based on sequences of row labels which correspond to roles (header, data, aggregate, etc). The papers [5,16,28,29] focus on the structural analysis for hierarchical headers in the stub. Koci et al. [30] propose the classification approach to identify five typical functional blocks of tables, using a wide range of cell features presented in spreadsheets.

The methods listed above are based on various assumptions about layout, style and textual content of widespread tables. Typically, they implement the assumptions as rules or heuristics incorporated in their algorithms. We think that there are many specific features beyond the types covered by these methods. Our table model does not determine any mandatory functional regions like head, stub, stubhead, or body. We also do not bind a cell with a role (heading or data value). Content that forms an entry, label, or category name can be located anywhere in a table. Instead of determining a few popular table types in algorithms, we propose to specify their features via external rules expressed in our domain-specific language.

The methods for table interpretation are mainly knowledge-based. They try to bind text in tables with some external concepts, using the following techniques: extraction ontologies [33], data frames [34], databases with facts (classes and relations) automatically collected from the Web [35], Linked Open Data like YAGO [36], WIKITOLOGY [37], FREEBASE [38], DBPEDIA [39], or general purpose knowledge taxonomy, PROBASE [40]. There are also several studies that propose to use contextual information that surrounds tables [7,41–43].

These methods rely on textual content of tables and their content but neglect their style features. In practice tabular data can be expressed via not only text but also style (e.g. different fonts or colors in cells can also have interpretable meanings). In contrast to them, our tool enables to specify and use both layout and style features. Our work slightly concerns table interpretation. We believe that many of the mentioned techniques can allow to extend our tool for annotating and conceptualization of table content.

### 7.2. Tabular data extraction and transformation tools

The methods [44–50] for generating linked data in RDF or OWL formats from spreadsheets or web tables include table analysis and

<sup>15</sup> <http://web.eecs.umich.edu/~michjc/structuredweb/index.html>.

<sup>16</sup> <https://www.w3.org/RDF>.

<sup>17</sup> <https://www.w3.org/OWL>.

interpretation. Some of them [45,46] work with various layouts of tables that differ from relational tables and propose domain-specific languages for specifying mappings of their data into structured representations. Langegger and Wölß [45] present XLWRAP, a spreadsheet-to-RDF wrapper where mappings are specified by XLWRAP expressions. O'Connor et al. [46] describe M<sup>2</sup>, a mapping language for converting data from arbitrary tables presented in spreadsheets to OWL format. Our aim in contrast to the mentioned techniques consists in generating only relational tables.

Data transformation tools typically deal only with tables, which have a simple “matrix” layout, where there are no merged cells, hierarchical headers, or footnotes. POTTER’S WHEEL [51], WRANGLER [52], and OPENREFINE<sup>18</sup> supply own domain-specific languages for cleaning messy values and reformatting tabular structures. HAEXCEL framework enables bidirectional mapping between a spreadsheet and a relational database [53]. MDSHEET framework also implements a technique that automatically infers relational schemes from spreadsheets [54]. The methodology [55] based on programming by examples includes a domain-specific language and algorithms for synthesis of table layout and content transformations. One of the latest work [56] develops FOFAH, a system to synthesize data transformation programs by examples. In contrast to the mentioned techniques, we work with arbitrary tables with complex layout. Our tool transforms them to relational tables and ETL (Extract, Transform, Load) tools can be used in further data normalization and cleansing.

Hung et al. [57] propose TRANSHEET, a spreadsheet-like formula language for specifying mappings between source spreadsheet data and a target schema. Their language is expressive and flexible for supporting practical spreadsheet-based data transformation in many cases. However, the mappings strictly determine correspondences between absolute cell addresses of source data and a target schema. Our language supports both absolute and relative cell addressing in comparison to TRANSHEET. Moreover, it allows formulating conditions for selecting cells through text and style features.

Barowy et al. [58] introduce the extraction language FLARE that extends regular expressions with geometric constructs. Interpretation of FLARE constraints allows extracting and transforming tabular data from spreadsheets. Their tool, FLASHRELATE, synthesizes a program in FLARE by a small set of examples (output relational tuples). The earlier study [59] presents TABLEPROG, a language that can express practical transformations over tabular data in spreadsheets. It is also designed for end-user programming by examples (input and output tables). Unlike them, we express transformations as table analysis and interpretation rules.

Chen et al. [60] focus on the problem of spreadsheet property (e.g. aggregation rows or hierarchical header) detection, identifying when a corresponding transformation program should be applied. They aim to convert any kind of spreadsheet data into relational tables. Their framework constructs trained property detectors based on rule-assisted active learning. It also uses crude user-provided rules on how a property can be detected for generating training data.

The recent works [61,62] develop domain-specific solutions. de Vos et al. [61] propose algorithms and accompanying software for automatic annotation of natural science spreadsheet tables. They combine structural properties of the tables (basic assumptions) and external vocabularies. Their algorithms implement a domain-specific set of classification rules and heuristics on these properties. The other tool [62] extracts RDF data from french government statistical spreadsheets and populates instances of their conceptual model. Both of these works limit ranges of processable tables by the considering domains.

To the best of our knowledge, there are no other domain-specific languages for tabular data transformation in terms of table understanding. Our language is based on the well-known terminology of Wang’s model [10]. In contrast to the existing mapping languages, we draw up this process as consecutive steps: role analysis, structural analysis, and interpretation.

### 7.3. Our previous work

We propose an approach to table understanding based on executing rules for table analysis and interpretation with a business rule engine [63]. We briefly introduce the preliminary version of our domain-specific rule language first in [64]. The prototype of our tool for canonicalization of arbitrary tables in spreadsheets is discussed in [12]. This paper combines and significantly expands our previous results. We explain in details our language in terms of the stages of table understanding. Moreover, this work revises and improves its syntax. We consider the original experiment application for transforming arbitrary tables of the same genre (government statistical tables). The performance evaluation has been done automatically for both role and structural analysis with the prepared ground-truth data first.

## 8. Conclusions

The work shows new possibilities in spreadsheet data transformation from arbitrary to relational tables based on rule-based programming for the table understanding stages.

The presented two-layered table object model combines the physical and logical table structure. Our model supports common layout and style spreadsheet features. Unlike others, it is not based on using functional cell regions. The functional items can be placed anywhere in a table. Therefore, the model can represent any table type satisfying the spreadsheet layout constraints. Another difference from existed representations is that the model provides data provenance for recovered semantics.

CRL, our domain-specific language, enables to express table analysis and interpretation rules. In contrast to the existing mapping languages, we draw up this process as consecutive steps: role analysis, structural analysis, and interpretation. There are no other domain-specific languages for tabular data transformation in terms of table understanding. Although, a general-purpose rule language can also express table analysis and interpretation rules. However, our language allows focusing on the logic of table understanding without excessive details. CRL defines a set of the essential actions for table understanding stages.

TABBYXL, our tool for spreadsheet data canonicalization, implements both the model and the language. The experiment demonstrates that the tool can be used for developing programs for transformation of spreadsheet data into the relational form. One rule-set can process a wide range of tables of the same genre, e.g. government statistical websites. Our tool can be used for populating databases from arbitrary tables, which share common features. It also can serve as a part of data extraction from tables (e.g. in tabular document processing [65]).

The work focuses on rather table analysis than issues of interpretation. We only recover categories as sets of labels, without binding it with a taxonomy of concepts. The further work on table content conceptualization can overcome this limitation. Moreover, we observe that arbitrary tables can contain messy (e.g. non-standardized values or typos) and useless (e.g. aggregations or padding characters) data. It seems to be interesting for the further work to incorporate additional techniques of data cleansing in our tool. Another development direction is to integrate the presented results with tools for extracting tables from documents (e.g. un-

<sup>18</sup> <http://openrefine.org>.



tagged PDF documents [66]) in end-to-end systems for table understanding.

## Acknowledgments

This work was supported by Russian Foundation for Basic Research [grant number 16-57-44034]; Russian Academy of Sciences [program number I.33]; and Council for Grants of the President of Russian Federation [grant number NSH-8081.2016.9]. The presented software was developed with resources of the Shared Equipment Center of Integrated Information and Computing Network of Irkutsk Research and Educational Complex.<sup>19</sup>

## References

- [1] M.J. Cafarella, A. Halevy, D.Z. Wang, E. Wu, Y. Zhang, Webtables: exploring the power of tables on the web, *Proc. VLDB Endow.* 1 (1) (2008) 538–549, doi:10.14778/1453856.1453916.
- [2] J. Eberius, K. Braunschweig, M. Hentsch, M. Thiele, A. Ahmadov, W. Lehner, Building the dresden web table corpus: A classification approach, in: *Proc. IEEE/ACM 2nd Int. Symposium on Big Data Computing*, 2015, pp. 41–50, doi:10.1109/BDC.2015.30.
- [3] O. Lehmberg, D. Ritze, R. Meusel, C. Bizer, A large public corpus of web tables containing time and context metadata, in: *Proc. 25th Int. Conf. Companion on World Wide Web*, 2016, pp. 75–76, doi:10.1145/2872518.2889386.
- [4] E. Crestan, P. Pantel, Web-scale table census and classification, in: *Proc. 4th ACM Int. Conf. Web Search and Data Mining*, 2011, pp. 545–554, doi:10.1145/1935826.1935904.
- [5] Z. Chen, M. Cafarella, Automatic web spreadsheet data extraction, in: *Proc. 3rd Int. Workshop Semantic Search Over the Web*, 2013, pp. 1:1–1:8, doi:10.1145/2509908.2509909.
- [6] L.R. Lautert, M.M. Scheidt, C.F. Dorneles, Web table taxonomy and formalization, *SIGMOD Rec.* 42 (3) (2013) 28–33, doi:10.1145/2536669.2536674.
- [7] K. Braunschweig, Recovering the semantics of tabular web data, *Technischen Universität Dresden, Dresden, Germany*, 2015 Ph.D. thesis.
- [8] M. Hurst, Layout and language: challenges for table understanding on the web, in: *Proc. 1st Int. Workshop Web Document Analysis*, 2001, pp. 27–30.
- [9] A. e Silva, A. Jorge, L. Torgo, Design of an end-to-end method to extract information from tables, *Int. J. Doc. Anal. Recognit.* 8 (2) (2006) 144–171, doi:10.1007/s10032-005-0001-x.
- [10] X. Wang, Tabular Abstraction, Editing, and Formatting, Ph.D. thesis, University of Waterloo, Waterloo, Ontario, Canada, 1996.
- [11] G. Nagy, Learning the characteristics of critical cells from web tables, in: *Proc. 21st Int. Conf. Pattern Recognition*, 2012, pp. 1554–1557.
- [12] A. Shigarov, V. Paramonov, P. Belykh, A. Bondarev, Rule-based canonicalization of arbitrary tables in spreadsheets, in: *Proc. 22nd Int. Conf. Information and Software Technologies*, 2016, pp. 78–91, doi:10.1007/978-3-319-46254-7\_7.
- [13] A. Shigarov, TabbyXL: Experiment data, (Mendeley Data, v1), 2017. doi:10.17632/448jdx7gcr.1.
- [14] G. Nagy, TANGO-DocLab web tables from international statistical sites (Troy\_200), 1, ID: Troy\_200\_1, 2016, [http://tc11.cvc.uab.es/datasets/Troy\\_200\\_1](http://tc11.cvc.uab.es/datasets/Troy_200_1).
- [15] D.W. Embley, M.S. Krishnamoorthy, G. Nagy, S. Seth, Converting heterogeneous statistical tables on the web to searchable databases, *Int. J. Doc. Anal. Recognit.* 19 (2) (2016) 119–138, doi:10.1007/s10032-016-0259-1.
- [16] Z. Chen, M. Cafarella, Integrating spreadsheet data via accurate and low-effort extraction, in: *Proc. 20th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2014, pp. 1126–1135, doi:10.1145/2623330.2623617.
- [17] Z. Chen, Information extraction on para-relational data, Ph.D. thesis, University of Michigan, US, 2016.
- [18] A. Pivk, P. Cimiano, Y. Sure, M. Gams, V. Rajković, R. Studer, Transforming arbitrary tables into logical form with TARTAR, *Data Knowl. Eng.* 60 (3) (2007) 567–595, doi:10.1016/j.datak.2006.04.002.
- [19] R. Abraham, M. Erwig, UCheck: a spreadsheet type checker for end users, *J. Visual Lang. Comput.* 18 (1) (2007) 71–95, doi:10.1016/j.jvlc.2006.06.001.
- [20] Y.-S. Kim, K.-H. Lee, Extracting logical structures from HTML tables, *Comput. Stand. Interfaces* 30 (5) (2008) 296–308, doi:10.1016/j.csi.2007.08.006.
- [21] C. Tao, D.W. Embley, Automatic hidden-web table interpretation, conceptualization, and semantic annotation, *Data Knowl. Eng.* 68 (7) (2009) 683–703, doi:10.1016/j.datak.2009.02.010.
- [22] S. Seth, G. Nagy, Segmenting tables via indexing of value cells by table headers, in: *Proc. 12th Int. Conf. Document Analysis and Recognition*, 2013, pp. 887–891, doi:10.1109/ICDAR.2013.181.
- [23] N.D. Mauro, F. Esposito, S. Ferilli, Finding critical cells in web tables with SRL: Trying to uncover the devil's tease, in: *Proc. 12th Int. Conf. on Document Analysis and Recognition*, 2013, pp. 882–886, doi:10.1109/ICDAR.2013.180.
- [24] M.D. Adelfio, H. Samet, Schema extraction for tabular data on the web, *Proc. VLDB Endow.* 6 (6) (2013) 421–432, doi:10.14778/2536336.2536343.
- [25] Z. Chen, M. Cafarella, J. Chen, D. Prevo, J. Zhuang, Senbazuru: a prototype spreadsheet database management system, *Proc. VLDB Endow.* 6 (12) (2013) 1202–1205, doi:10.14778/2536274.2536276.
- [26] D.W. Embley, S. Seth, G. Nagy, Transforming web tables to a relational database, in: *Proc. 22nd Int. Conf. Pattern Recognition*, 2014, pp. 2781–2786, doi:10.1109/ICPR.2014.479.
- [27] G. Nagy, D. Embley, S. Seth, End-to-end conversion of HTML tables for populating a relational database, in: *Proc. 11th IAPR Int. Workshop Document Analysis Systems*, 2014, pp. 222–226, doi:10.1109/DAS.2014.9.
- [28] R. Rastan, H.-y. Paik, J. Shepherd, A. Haller, Automated table understanding using stub patterns, pp. 533–548, doi:10.1007/978-3-319-32025-0\_33.
- [29] K. Goto, Y. Ohta, H. Inakoshi, N. Yugami, Extraction algorithms for hierarchical header structures from spreadsheets, in: *Proc. Workshops of the EDBT/ICDT 2016 Joint Conference*, 2016.
- [30] E. Koci, M. Thiele, O. Romero, W. Lehner, A machine learning approach for layout inference in spreadsheets, in: *Proc. 8th Int. Joint Conf. Knowledge Discovery, Knowledge Engineering and Knowledge Management*, 2016, pp. 77–88, doi:10.5220/0006052200770088.
- [31] G. Nagy, D.W. Embley, M. Krishnamoorthy, S. Seth, Clustering header categories extracted from web tables, *Proc. SPIE 9402* (2015) 94020M–94020M–12, doi:10.1117/12.2076209.
- [32] G. Nagy, S. Seth, Table headers: an entrance to the data mine, in: *Proc. 23rd Int. Conf. Pattern Recognition*, 2016, pp. 4065–4070, doi:10.1109/ICPR.2016.7900270.
- [33] D. Embley, C. Tao, S. Liddle, Automating the extraction of data from HTML tables with unknown structure, *Data Knowl. Eng.* 54 (1) (2005) 3–28, doi:10.1016/j.datak.2004.10.004.
- [34] Y. Tijerino, D. Embley, D. Lonsdale, Y. Ding, G. Nagy, Towards ontology generation from tables, *World Wide Web* 8 (3) (2005) 261–285, doi:10.1007/s11280-005-0360-8.
- [35] P. Venetis, A. Halevy, J. Madhavan, M. Pasca, W. Shen, F. Wu, G. Miao, C. Wu, Recovering semantics of tables on the web, *Proc. VLDB Endow.* 4 (9) (2011) 528–538, doi:10.14778/2002938.2002939.
- [36] G. Limaye, S. Sarawagi, S. Chakrabarti, Annotating and searching web tables using entities, types and relationships, *Proc. VLDB Endow.* 3 (1–2) (2010) 1338–1347, doi:10.14778/1920841.1921005.
- [37] V. Mulwad, T. Finin, Z. Syed, A. Joshi, Using linked data to interpret tables, in: *Proc. 1st Int. Conf. Consuming Linked Data*, 2010, pp. 109–120. <http://dl.acm.org/citation.cfm?id=2878947.2878957>.
- [38] D. Deng, Y. Jiang, G. Li, J. Li, C. Yu, Scalable column concept determination for web tables using large knowledge bases, *Proc. VLDB Endow.* 6 (13) (2013) 1606–1617, doi:10.14778/2536258.2536271.
- [39] E. Muñoz, A. Hogan, A. Mileo, Using linked data to mine RDF from wikipedia's tables, in: *Proc. 7th ACM Int. Conf. Web Search and Data Mining*, 2014, pp. 533–542, doi:10.1145/2556195.2556266.
- [40] J. Wang, H. Wang, Z. Wang, K.Q. Zhu, Understanding tables on the web, in: *Proc. 31st Int. Conf. Conceptual Modeling*, 2012, pp. 141–155, doi:10.1007/978-3-642-34002-4\_11.
- [41] V. Govindaraju, C. Zhang, C. Ré, Understanding tables in context using standard NLP toolkits, in: *Proc. 51st Annual Meeting of the Association for Computational Linguistics*, 2: Short Papers, 2013, pp. 658–664.
- [42] Z. Zhang, Towards Efficient and Effective Semantic Table Interpretation, pp. 487–502, doi:10.1007/978-3-319-11964-9\_31.
- [43] M. Yoshida, K. Matsumoto, K. Kita, Table topic models for hidden unit estimation, in: *Proc. 12th Asia Information Retrieval Societies Conference*, 2016, pp. 302–307, doi:10.1007/978-3-319-48051-0\_23.
- [44] L. Han, T. Finin, C. Parr, J. Sachs, A. Joshi, RDF123: From Spreadsheets to RDF, 451–466, doi:10.1007/978-3-540-88564-1\_29.
- [45] A. Langegger, W. WöB, XLWrap – Querying and Integrating Arbitrary Spreadsheets with SPARQL, 359–374, doi:10.1007/978-3-642-04930-9\_23.
- [46] M.J. O'Connor, C. Halaschek-Wiener, M.A. Musen, Mapping Master: A Flexible Approach for Mapping Spreadsheets to OWL, 194–208, doi:10.1007/978-3-642-17749-1\_13.
- [47] V. Mulwad, T. Finin, A. Joshi, A Domain Independent Framework for Extracting Linked Semantic Data from Tables, 16–33, doi:10.1007/978-3-642-34213-4\_2.
- [48] M. Fiorelli, T. Lorenzetti, M.T. Pazienza, A. Stellato, A. Turbati, Sheet2RDF: a flexible and dynamic spreadsheet import&lifting framework for RDF, in: *Proc. 28th Int. Conf. Industrial, Engineering and Other Applications of Applied Intelligent Systems*, 2015, pp. 131–140, doi:10.1007/978-3-319-19066-2\_13.
- [49] M. Galkin, D. Mourmteyev, S. Auer, Identifying web tables: supporting a neglected type of content on the web, in: *Proc. 6th Int. Conf. Knowledge Engineering and Semantic Web*, 2015, pp. 48–62, doi:10.1007/978-3-319-24543-0\_4.
- [50] I. Ermilov, A.-C. N. Ngomo, TAIPAN: Automatic Property Mapping for Tabular Data, 163–179, doi:10.1007/978-3-319-49004-5\_11.
- [51] V. Raman, J.M. Hellerstein, Potter's wheel: an interactive data cleaning system, in: *Proc. the 27th Int. Conf. Very Large Data Bases*, 2001, pp. 381–390. <http://dl.acm.org/citation.cfm?id=645927.672045>.
- [52] S. Kandel, A. Paepcke, J. Hellerstein, J. Heer, Wrangler: interactive visual specification of data transformation scripts, in: *Proc. SIGCHI Conf. Human Factors in Computing Systems*, 2011, pp. 3363–3372, doi:10.1145/1978942.1979444.
- [53] J. Cunha, J.A. Saraiva, J. Visser, From spreadsheets to relational databases and back, in: *Proc. ACM SIGPLAN Workshop Partial Evaluation and Program Manipulation*, 2009, pp. 179–188, doi:10.1145/1480945.1480972.
- [54] J. Cunha, M. Erwig, J. Mendes, J. Saraiva, Model inference for spreadsheets, *Autom Softw Eng* 23 (3) (2016) 361–392, doi:10.1007/s10515-014-0167-x.

<sup>19</sup> <http://net.icc.ru>.



- [55] S. Gulwani, W.R. Harris, R. Singh, Spreadsheet data manipulation using examples, *Commun. ACM* 55 (8) (2012) 97–105, doi:[10.1145/2240236.2240260](https://doi.org/10.1145/2240236.2240260).
- [56] Z. Jin, M.R. Anderson, M. Cafarella, H.V. Jagadish, Foofah: transforming data by example, in: *Proc. ACM Int. Conf. Management of Data*, 2017, pp. 683–698, doi:[10.1145/3035918.3064034](https://doi.org/10.1145/3035918.3064034).
- [57] V. Hung, B. Benatallah, R. Saint-Paul, Spreadsheet-based complex data transformation, in: *Proc. 20th ACM Int. Conf. Information and Knowledge Management*, 2011, pp. 1749–1754, doi:[10.1145/2063576.2063829](https://doi.org/10.1145/2063576.2063829).
- [58] D.W. Barowy, S. Gulwani, T. Hart, B. Zorn, FlashRelate: extracting relational data from semi-structured spreadsheets using examples, *SIGPLAN Not.* 50 (6) (2015) 218–228, doi:[10.1145/2813885.2737952](https://doi.org/10.1145/2813885.2737952).
- [59] W.R. Harris, S. Gulwani, Spreadsheet table transformations from examples, *SIGPLAN Not.* 46 (6) (2011) 317–328, doi:[10.1145/1993316.1993536](https://doi.org/10.1145/1993316.1993536).
- [60] Z. Chen, X. Rong, S. Dadiomov, R. Wesley, G. Xiao, D. Cory, M. Cafarella, J. Mackinlay, Spreadsheet Property Detection with Rule-assisted Active Learning, Technical Report, 2016, CSE-TR-601-16, <https://www.cse.umich.edu/techreports/cse/2016/CSE-TR-601-16.pdf>.
- [61] M. de Vos, J. Wielemaker, H. Rijgersberg, G. Schreiber, B. Wielinga, J. Top, Combining information on structure and content to automatically annotate natural science spreadsheets, *Int. J. Hum. Comput. Stud.* 103 (2017) 63–76, doi:[10.1016/j.ijhcs.2017.02.006](https://doi.org/10.1016/j.ijhcs.2017.02.006).
- [62] T.D. Cao, I. Manolescu, X. Tannier, Extracting linked data from statistic spreadsheets, in: *Proc. Int. Workshop Semantic Big Data*, 2017, pp. 5:1–5:5, doi:[10.1145/3066911.3066914](https://doi.org/10.1145/3066911.3066914).
- [63] A. Shigarov, Table understanding using a rule engine, *Expert Syst. Appl.* 42 (2) (2015) 929–937, doi:[10.1016/j.eswa.2014.08.045](https://doi.org/10.1016/j.eswa.2014.08.045).
- [64] A. Shigarov, Rule-based table analysis and interpretation, in: *Proc. 21st Int. Conf. Information and Software Technologies*, 2015, pp. 175–186, doi:[10.1007/978-3-319-24770-0\\_16](https://doi.org/10.1007/978-3-319-24770-0_16).
- [65] S. Yang, J. Guo, R. Wei, Semantic interoperability with heterogeneous information systems on the internet through automatic tabular document exchange, *Inf. Syst.* 69 (2017) 195–217, doi:[10.1016/j.is.2016.10.010](https://doi.org/10.1016/j.is.2016.10.010).
- [66] A. Shigarov, A. Mikhailov, A. Altaev, Configurable table structure recognition in untagged PDF documents, in: *Proc. ACM Symposium on Document Engineering*, 2016, pp. 119–122, doi:[10.1145/2960811.2967152](https://doi.org/10.1145/2960811.2967152).